プログラミング３ 講義資料

第5回 gitの実践：ブランチとマージ

- ブランチの役割
- merge と切り替えコマンド
- コンフリクトの発生と解消

目次

- ブランチの概念
- マージとコンフリクト

ブランチの概念

関連ドキュメント: この節の解説

今回の目的と到達目標

今回の目的

- ブランチの作成・切替ができる
- merge を安全に行える
- コンフリクトを正しく解消できる

到達目標

- [2] git の基本操作（ブランチ・マージ）

ブランチとは何か

- 並行する作業ラインを作る仕組み
- main は安定版、feature は作業中の変更を隔離
- 作業単位ごとにブランチを切れば、未完成の変更が main を壊さない
- HEAD = 現在いる位置を示すポインタ

状態	HEAD が指す先
main で作業	main 先頭
feature/foo で作業	feature/foo 先頭

ブランチ操作コマンド

```
git branch                # ブランチ一覧
git branch <name>         # ブランチを作成
git switch <name>         # ブランチを切り替え
git switch -c <name>      # 作成+切替を一度に
git branch -d <name>      # マージ済みブランチを削除
```

- `git switch` が新しい推奨コマンド（旧: `git checkout`）
- 切替前に作業中の変更はコミットまたは `stash` しておく

マージとコンフリクト

関連ドキュメント: この節の解説

マージの基本

```
git switch main
git merge feature/foo   # feature/foo を main に取り込む
```

パターンは2種類:

1. **Fast-forward** — main 先頭が後ろにあるだけなら、HEAD を進めるだけ
2. **3-way merge** — 双方が進んでいる場合、新しいマージコミットを作る

コンフリクトとは

- 同じファイルの同じ行を **別々のブランチで** 変更した場合に発生
- git は自動で統合できず、人間の判断を求める
- 該当ファイルに次のマーカーが挿入される:

```
<<<<<<< HEAD
main での内容
=======
feature での内容
>>>>>>> feature/foo
```

解消手順

1. `git status` で衝突ファイルを確認
2. エディタで該当ファイルを開く
3. `<<<<<<<` 〜 `>>>>>>>` マーカーを見て、どちらを残すか/両方混ぜるかを判断
4. マーカー行を削除して正しい内容に整える
5. `git add <file>` で解消済みを宣言
6. `git commit`（または `git merge --continue`）でマージ完了

やってはいけない操作

Tips

- ❌ `git reset --hard` を勢いで打たない — コミット未済の変更が消える
- ❌ マーカー行を残したままコミットしない
- ❌ 他人のブランチを勝手に `force push` しない
- ✅ 迷ったら `git status` → 作業ツリーを確認してから次の操作へ

今回のまとめ

- ブランチは並行作業の隔離、main を安全に保つための仕組み
- `git switch -c` で作成+切替、`git merge` で統合
- コンフリクトはマーカーを見て手動解消 → `git add` → `git commit`

今回カバーした MCC 項目

- V-D-1 プログラミング

次回予告

- 第6回: GitHub とリモートリポジトリ — push / pull / clone